

DATA301-DATA471: Data Science in Practice

Assignment 4

Leon Cai (300692424)

Victoria University of Wellington
September 2026

Exercise 1

1.1 Order statistics for the quartiles [10%]

Summary of the two articles. Nine common ways to compute a sample quantile are compared in Wicklin (2017), and they come from Hyndman and Fan (1996) (H&F). All nine have the same form, $\hat{Q}(p) = (1 - \gamma)x_{(j)} + \gamma x_{(j+1)}$, where $x_{(j)}$ and $x_{(j+1)}$ are the two order statistics next to p and γ is a weight between them. Both j and γ depend on p , the sample size n , and a number m that is different for each definition ($j = \lfloor np + m \rfloor$). Types 1-3 jump between values instead of changing smoothly (a staircase in the article), and they always give a value that is already in the data, or the average of two neighbours. Types 4-9 interpolate between neighbouring order statistics, so the answer can fall between two data values. The nine agree closely for larger samples (the article uses $n = 100$) but can differ more for small samples or when there are big gaps between neighbouring values, which includes the $n = 5$ cells in the Monte Carlo comparison below.

H&F also checked each definition against six properties a good sample quantile should have, for example continuity in p and agreeing with the usual median at $p = 0.5$. Type 5 satisfies all six and types 6-9 satisfy five. Of these, H&F recommend type 8, because its estimate is approximately median-unbiased whatever the distribution is (about as likely to be too high as too low). Types 6 and 7 do not have this property, and type 9 has it only for Normal data.

Choice. I use type 8 throughout (it is the default in `lambda_iqr()` and everywhere that function is called). One reason is H&F's recommendation. The other two are that the Exponential distribution is skewed, so type 9, which is made for Normal data, does not fit it, and that the smallest sample size in the design, $n = 5$, is where the choice matters most.

$\hat{\lambda}_{IQR}$ is also reused in the bootstrap correction and in every Monte Carlo replication, so a bad quantile definition would affect all of those results.

Default in R. R's `quantile()` uses type 7 by default, checked below. R follows S-Plus, which used type 7 (Hyndman and Fan (1996), Section 4). So the R default is not the type 8 that H&F recommend.

```
quantile(1:10)                                # uses R's default type
##      0%      25%      50%      75%     100%
##  1.00   3.25   5.50   7.75  10.00
identical(quantile(1:10), quantile(1:10, type = 7)) # confirms default = type 7
## [1] TRUE
```

1.2 Bootstrap bias-corrected estimator [20%]

Derivation. If we knew the true $\text{Exponential}(\lambda)$ distribution, we could estimate the bias of $\hat{\lambda}_{IQR}$ by simulating many samples from it and comparing the average of $\hat{\lambda}_{IQR}$ with the true λ , as in the Week 9 lecture. In practice we only have one sample $\mathbf{X}$ and do not know λ . The bootstrap gets around this by treating $\mathbf{X}$ as if it were the population, and $\hat{\lambda}_{IQR}(\mathbf{X})$ as if it were the true value. For a fixed sample $\mathbf{X}$ of size n , draw B bootstrap resamples $\mathbf{X}_1^*, \dots, \mathbf{X}_B^*$ (each one has n values sampled from $\mathbf{X}$ with replacement). The bootstrap estimate of the bias of $\hat{\lambda}_{IQR}$ is

$$\widehat{\text{bias}} = \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}(\mathbf{X}_b^*) - \hat{\lambda}_{IQR}(\mathbf{X}).$$

The first-order bias-corrected estimator subtracts this estimated bias:

$$\tilde{\lambda}(\mathbf{X}) = \hat{\lambda}_{IQR}(\mathbf{X}) - \widehat{\text{bias}} = 2\hat{\lambda}_{IQR}(\mathbf{X}) - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}(\mathbf{X}_b^*).$$

```
lambda_ml <- function(x) 1 / mean(x)

lambda_iqr <- function(x, type = 8) {
  q <- quantile(x, probs = c(0.25, 0.75), type = type)
  log(3) / (q[[2]] - q[[1]])
}

lambda_tilde <- function(x, B = 200, type = 8) {
  n <- length(x)
  boot_est <- replicate(B, lambda_iqr(sample(x, n, replace = TRUE),
                                     type = type))
  # Small n: a bootstrap resample can tie at the quartiles (q3=q1), making
  # lambda_iqr(.) = Inf. Drop these before averaging so one Inf doesn't
  # dominate the correction.
  boot_est <- boot_est[is.finite(boot_est)]
  2 * lambda_iqr(x, type = type) - mean(boot_est)
}

x <- rexp(30, rate = 2)
round(c(ML = lambda_ml(x), IQR = lambda_iqr(x),
        bias_corrected = lambda_tilde(x)), 3)
##           ML           IQR bias_corrected
##          1.563          1.741          1.814
```

For type 8 at $n = 5$, a resample only gets $q_{3/4} = q_{1/4}$ (and is dropped) when all five draws are the same value, which has probability $5 \times (1/5)^5 = 0.0016$, so about 0.3 of the 200 resamples per sample. For $n = 30$ and $n = 100$ it essentially never happens.

1.3 Monte Carlo comparison [20%]

Design. This follows the course's Monte Carlo tutorial (Frery 2026): for each situation $(\lambda, n) \in \Lambda \times N$, R independent replications each give one sample $\mathbf{X}$ of size n and one value of each estimator, and the MSE is the average of $(\hat{\lambda} - \lambda)^2$ over the R replications.

I use $R = 5000$ for every situation, as suggested in the Week 9 lecture. The `rel_se` column in Table 1 checks this: for each case it is the biggest, over the three estimators, of the Monte

Carlo standard error of the MSE divided by the MSE. It is only 2-3.5% for $n = 30$ and $n = 100$, but 70-90% for $n = 5$, where a few extreme replications dominate the averages (most of all for `bias_corrected`). So the $n = 5$ values are rough. The huge `bias_corrected` gap there is safe from that, but the other comparisons are better read from $n = 30$ and $n = 100$. Using the same R everywhere keeps the cells comparable.

```
mc_mse <- function(lambda, n, R, B = 200, type = 8) {
  est <- matrix(NA_real_, nrow = R, ncol = 3,
               dimnames = list(NULL, c("ML", "IQR", "bias_corrected")))
  for (r in seq_len(R)) {
    x <- rexp(n, rate = lambda)
    est[r, ] <- c(lambda_ml(x), lambda_iqr(x, type = type),
                 lambda_tilde(x, B = B, type = type))
  }
  sq_err <- (est - lambda)^2
  rel_se <- max(apply(sq_err, 2, sd) / sqrt(R) / colMeans(sq_err))
  c(colMeans(sq_err), rel_se = rel_se)
}

lambda_grid <- c(0.1, 1, 10)
n_grid <- c(5, 30, 100)
R <- 5000

results <- do.call(rbind, lapply(lambda_grid, function(lam) {
  do.call(rbind, lapply(n_grid, function(n) {
    mse <- mc_mse(lam, n, R = R)
    data.frame(lambda = lam, n = n, R = R, t(mse), check.names = FALSE)
  }))
}))
```

Table 1: MSE of the three estimators for each (λ, n) , $R = 5000$, and the largest relative standard error `rel_se`.

lambda	n	ML	IQR	bias_corrected	rel_se
0.1	5	6.35e-03	1.26e-02	1.87e+01	0.860
0.1	30	4.00e-04	1.00e-03	1.24e-03	0.035
0.1	100	1.08e-04	2.36e-04	2.79e-04	0.024
1.0	5	5.68e-01	9.37e-01	4.75e+03	0.904
1.0	30	3.78e-02	9.54e-02	1.19e-01	0.031
1.0	100	1.07e-02	2.30e-02	2.69e-02	0.024
10.0	5	6.18e+01	9.64e+01	7.00e+04	0.692
10.0	30	4.01e+00	9.75e+00	1.23e+01	0.034
10.0	100	1.08e+00	2.45e+00	2.82e+00	0.027

Figure 1 plots the same numbers on log-log axes, since `bias_corrected`'s MSE spans several orders of magnitude.

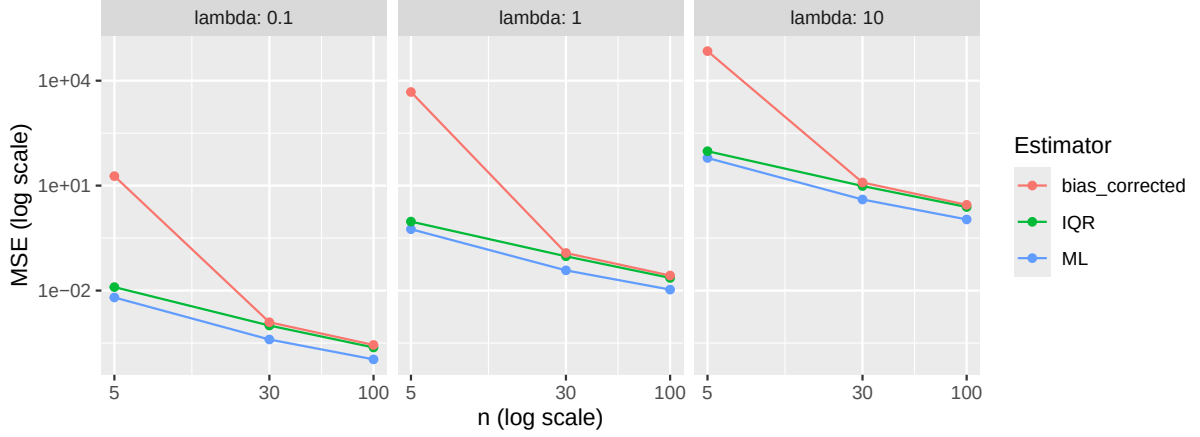


Figure 1: MSE of the three estimators across the (λ, n) design, log-log scale.

Discussion. $\hat{\lambda}_{ML}$ has the lowest MSE in every one of the nine cells, which is what I expected: it is the maximum likelihood estimator and uses the whole sample, while $\hat{\lambda}_{IQR}$ only uses two order statistics. The MSE gap stays in a similar range across the design: $\hat{\lambda}_{IQR}$'s MSE is roughly 1.6 to 2.5 times $\hat{\lambda}_{ML}$'s at every (λ, n) , with no clear trend in n or λ . This roughly fits the idea that the IQR estimator loses a fixed share of efficiency because it throws away most of the sample, although that is only guaranteed as $n \rightarrow \infty$. The spread in the ratio is bigger than the Monte Carlo error at $n = 30$ and $n = 100$, so part of it is real finite-sample behaviour, not just noise.

The bias-corrected estimator $\tilde{\lambda}$ is *worse* than the uncorrected $\hat{\lambda}_{IQR}$ at every (λ, n) in the design, not only at $n = 5$. At $n = 100$ the correction makes the MSE about 15-18% bigger, at $n = 30$ about 24-26% bigger, and at $n = 5$ it is much worse: about 700 to 5000 times bigger.

I think the reason is a bias-variance trade-off that goes the wrong way. $\hat{\lambda}_{IQR}$ is a reciprocal of the sample's interquartile spread, so it is nonlinear. A bootstrap resample can, by chance, have a smaller or larger spread than the original sample. After taking the reciprocal and doubling it in $\tilde{\lambda} = 2\hat{\lambda}_{IQR} - \overline{\hat{\lambda}_{IQR}^*}$, this small difference becomes a big swing in the correction term. The variance this adds is larger than the bias it removes, so the MSE gets worse, and more so when n is small, because bootstrap resamples are then more likely to have an unusually small spread. This is only my explanation, since I did not check bias and variance separately, and it only covers the correction that resamples $\mathbf{X}$ with replacement, as I did here, not a parametric bootstrap.

The MSE of $\hat{\lambda}_{ML}$ and $\hat{\lambda}_{IQR}$ goes down as n grows, roughly like $1/n$ between $n = 30$ and $n = 100$ (faster from $n = 5$ to $n = 30$), and it goes up roughly with λ^2 : at $n = 5$, $\text{MSE}(\hat{\lambda}_{ML})/\lambda^2$ is about 0.64, 0.57, and 0.62 at $\lambda = 0.1, 1, 10$. That is what a rate should do under a change of time unit. The bias-corrected estimator does not follow this at $n = 5$.

So if the Exponential model is believed, $\hat{\lambda}_{ML}$ is the one to use here. $\hat{\lambda}_{IQR}$ is a simpler alternative that costs about twice the MSE, though I did not test it with outliers. The bootstrap correction I used is not worth it for this estimator.

Exercise 2

`ausbeer` comes with the `fpp2` package. It is quarterly beer production data for Australia. Figure 2 shows the full series; the sections on frequency, trend, seasonality, variance and unusual observations all refer back to this same series, some directly from the plot and some from numbers computed later.

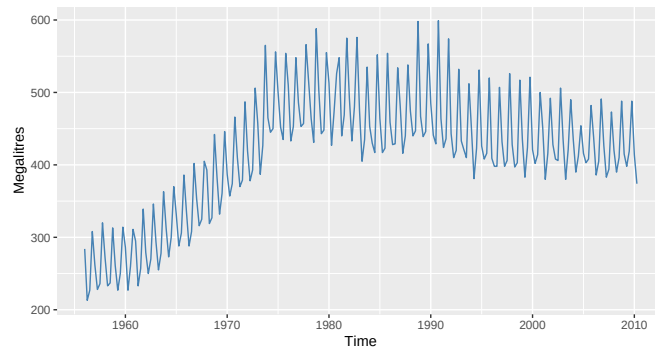


Figure 2: Quarterly Australian beer production, 1956 Q1–2010 Q2.

2.1 What is the frequency of the series? How can you determine this from the data and/or the time-series plot?

The frequency is 4 (quarterly data), and the series runs from 1956 Q1 to 2010 Q2, confirmed below by `frequency()`, `start()` and `end()`.

```
frequency(ausbeer)
## [1] 4
start(ausbeer)
## [1] 1956 1
end(ausbeer)
## [1] 2010 2
```

Counting cycles by eye on the full 54-year series in Figure 2 is hard, since each year only takes up a few pixels, but zooming into a handful of years in Figure 3 makes the four-per-year pattern countable directly:

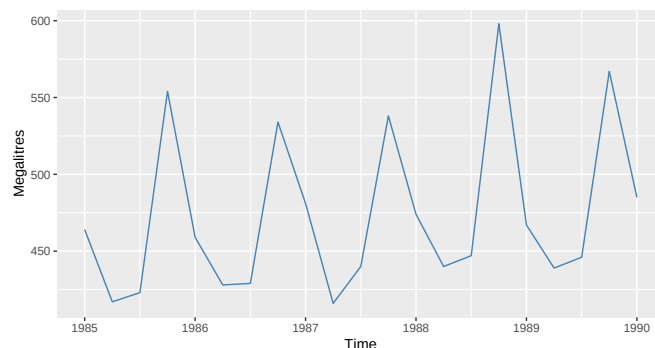


Figure 3: `ausbeer` zoomed into 1985–1990, showing four quarters per year.

2.2 Describe the overall behaviour of the series. Is there evidence of a long-term trend?

There isn't one single trend here, more like three phases. Production rises steadily from 1956 to the mid-1970s (about 260 to 485 megalitres, nearly doubling), stays high and fluctuates until about 1990, then drops fairly sharply in the early 1990s (about 490 to 445 in three years) and levels off there through the 2000s.

To check this against a straight line without fitting any model, Figure 4 joins the early-period average (1956-60) to the late-period average (2005-10) with a red line. If the trend were linear, the yearly average (blue) would stay close to it. Instead it sits above the line from the mid-1960s to around 2000, by roughly 90-170 megalitres between 1970 and 1990, and only returns to it at the end. So the trend is not a straight line, and the blue line shows the three phases more clearly than the raw quarterly plot (Figure 2).

```
ausbeer_df <- data.frame(time = as.numeric(time(ausbeer)),
                        value = as.numeric(ausbeer))
yearly <- aggregate(value ~ floor(time), ausbeer_df, mean)
names(yearly) <- c("year", "value")
yearly <- yearly[yearly$year < 2010, ] # 2010 only has 2 quarters, drop it

early <- mean(ausbeer_df$value[ausbeer_df$time < 1961])
late <- mean(ausbeer_df$value[ausbeer_df$time >= 2005])
t_early <- mean(ausbeer_df$time[ausbeer_df$time < 1961])
t_late <- mean(ausbeer_df$time[ausbeer_df$time >= 2005])
ausbeer_df$line <- early +
  (late - early) * (ausbeer_df$time - t_early) / (t_late - t_early)
```

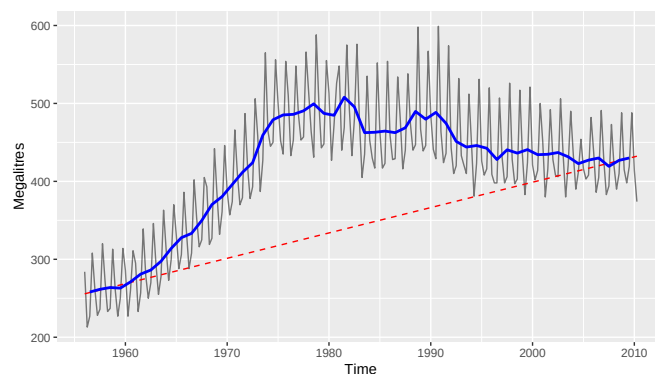


Figure 4: `ausbeer`'s yearly average (blue) against a straight line joining the early- and late-period averages (red).

2.3 Is there evidence of seasonality? If so, describe the seasonal pattern.

Yes, and it's a strong, regular pattern: every year has the same shape, low in Q2 and Q3 and high in Q4, confirmed by the quarter averages below. Q4 is the highest quarter and Q2 the lowest. This probably follows the Australian seasons, since Q4 (October to December) is spring going into summer and more beer is likely drunk then, but the data alone can't show that reason, it is only my guess.

```
quarter_means <- aggregate(as.numeric(ausbeer) ~ cycle(ausbeer), FUN = mean)
names(quarter_means) <- c("quarter", "mean")
quarter_means
```

```
##   quarter    mean
## 1         1 416.8182
## 2         2 372.6182
## 3         3 387.3704
## 4         4 485.4444
```

Looking at each of the 54 full years (2010 is incomplete), the highest quarter is Q4 in every single year, and the lowest is Q2 in 48 years and Q3 in the other six:

```
by_year <- data.frame(year = floor(as.numeric(time(ausbeer))),
                      quarter = as.numeric(cycle(ausbeer)),
                      value = as.numeric(ausbeer))
by_year <- by_year[by_year$year < 2010, ]
highest <- sapply(split(by_year, by_year$year),
                  function(d) d$quarter[which.max(d$value)])
lowest <- sapply(split(by_year, by_year$year),
                 function(d) d$quarter[which.min(d$value)])
table(highest)
## highest
## 4
## 54
table(lowest)
## lowest
## 2 3
## 48 6
```

Figure 5 agrees with this: Q4's box sits above Q2's and Q3's with no overlap at all, and only a small overlap with Q1's. The boxes pool all 54 years, so the long lower whiskers mostly come from the early low-production years and not from the seasons. The yearly counts above are the better evidence that Q4 is the high quarter every year.

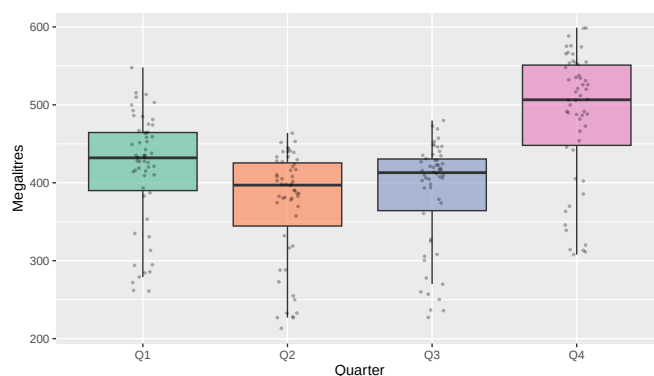


Figure 5: Distribution of `ausbeer` by quarter.

2.4 Does the variability appear constant over time? Explain what you see in the plot and whether a transformation might be appropriate.

Not really constant. In Figure 2 the seasonal swings look visibly tighter in the 2000s than in the 1970s-90s, even though the level is not dramatically lower. To put a number on it, I measured the swing in each full year as the highest quarter minus the lowest (this way the trend is not counted as variability), and averaged it over each period, next to the level and the swing as a share of the level (Table 2):

```

breaks <- c(1955, 1970, 1980, 1990, 2000, 2011)
labels <- c("1956-70", "1971-80", "1981-90", "1991-2000", "2001-09")
year_level <- tapply(by_year$value, by_year$year, mean)
year_swing <- tapply(by_year$value, by_year$year,
  function(x) max(x) - min(x))
period <- cut(as.numeric(names(year_swing)), breaks = breaks, labels = labels)
level <- tapply(year_level, period, mean)
swing <- tapply(year_swing, period, mean)
knitr::kable(data.frame(period = labels, level = round(level, 1),
  swing = round(swing, 1),
  share = round(swing / level, 2),
  row.names = NULL),
  format = "latex", booktabs = TRUE, linesep = "")

```

Table 2: Average level, average within-year swing (highest minus lowest quarter) and swing as a share of the level, by period.

period	level	swing	share
1956-70	310.2	99.5	0.32
1971-80	470.7	124.9	0.27
1981-90	478.3	136.4	0.29
1991-2000	443.8	122.9	0.28
2001-09	428.8	94.2	0.22

The swing grows from about 100 megalitres in 1956-70 to about 135 in 1981-90 while the level goes from about 310 to 480, so early on the two move together, which is the pattern a log transform is meant for. After that the swing falls back to about 95 in the 2000s while the level only drops to about 430. As a share of the level the swing stays around 0.27 to 0.29 from 1971 to 2000, is higher early (0.32) and lowest at the end (0.22). So the evidence for a log transform, where swings are supposed to grow in proportion to the level, is mixed: some support in the early decades, but it does not hold in the 2000s.

2.5 Are there any observations that you would regard as unusual? Identify them and explain why they attract attention.

Two things stand out. The first is that the whole period before about 1970 looks different from the rest: the level is much lower (about 310 against 430-480 later) and the swings are relatively bigger (0.32 of the level against 0.22-0.29), as shown above. The points below the whiskers in Figure 5 are all from 1956-59, so they belong to the same thing rather than being scattered noise. The forecasting examples in Hyndman and Athanasopoulos (2018) mostly use `window(ausbeer, start = 1992)`, which fits this: the early decades look like a different, still growing period.

The second is one single odd point: 1981 Q1 comes in at 548. I compared every quarter with the average of the same quarter in the three years on each side (no model involved), and this is the biggest gap anywhere in `ausbeer`: 56 megalitres above its neighbours, about 3.8 times the typical size of that gap:

```

ausbeer_df$quarter <- as.numeric(cycle(ausbeer))
local_gap <- rep(NA, nrow(ausbeer_df))
for (i in seq_len(nrow(ausbeer_df))) {
  same_quarter <- ausbeer_df$quarter == ausbeer_df$quarter[i]

```

```

close_in_time <- abs(ausbeer_df$time - ausbeer_df$time[i]) <= 3
neighbours <- same_quarter & close_in_time & seq_len(nrow(ausbeer_df)) != i
if (sum(neighbours) >= 4) {
  local_gap[i] <- ausbeer_df$value[i] - mean(ausbeer_df$value[neighbours])
}
}
ausbeer_df$time[which.max(abs(local_gap))]
## [1] 1981
max(abs(local_gap), na.rm = TRUE) / sd(local_gap, na.rm = TRUE)
## [1] 3.799203

```

2.6 Plot the series by season (for example, using a seasonal plot). What does this reveal that is less obvious in the ordinary time-series plot?

Figure 6 puts every year on its own line across the four quarters. Compared to the ordinary time plot (Figure 2), this makes a few things easier to see:

- Almost every year has the same up-down-up shape: climbing to a peak at Q4 without exception, and usually dipping lowest at Q2 (though Q3 is the lowest quarter in six of the 54 full years, like 1978 and 1990).
- The lines are stacked in the same order as the trend: the earliest years (orange) sit lowest, the lines climb through the 1970s-80s (green and blue), and the most recent years (pink) come back down.
- Comparing the same quarter across years is much easier here than in the ordinary time plot, where the years are spread left to right.
- The 1981 anomaly from section 2.5 shows up here too: its line starts at Q1 clearly above every neighbouring year's.

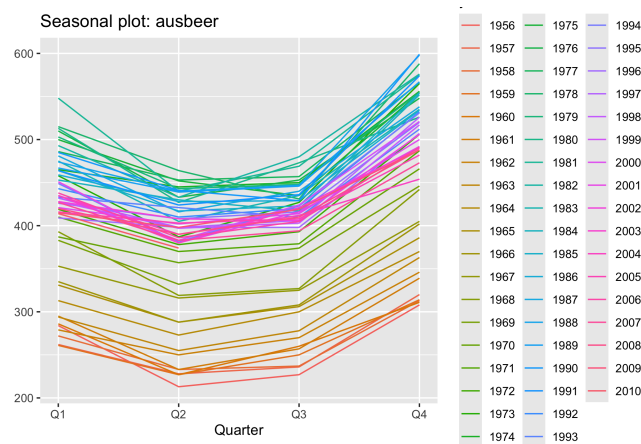


Figure 6: Seasonal plot of `ausbeer`, one line per year.

2.7 Plot the seasonal subseries. Which quarters have the highest and lowest typical values? Is the seasonal pattern stable over time?

Figure 7 has one panel per quarter, with each quarter's mean marked in red:

- Because the years stay in time order here (unlike the jittered points in Figure 5), each panel still follows its own rise, plateau and decline, like the overall trend. So the *level* within a quarter is not stable over time: the seasonal part and the trend are mixed together.
- It also confirms two things from section 2.3: Q4 has the highest typical value and Q2 the

lowest, and the seasonal *pattern* (which quarter is high and which is low) looks stable, not flipping or fading out in any panel.

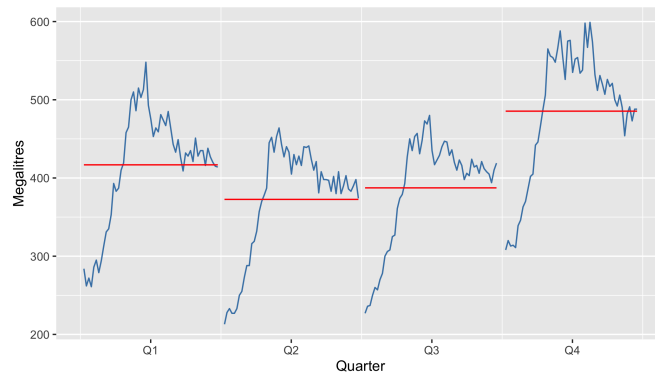


Figure 7: Seasonal subseries plot of `ausbeer`, one panel per quarter.

2.8 Would you detrend or transform the series before modelling it? Give a visual argument for your answer. Do not fit a model.

Detrending, yes. Figure 8 plots the gap between the raw series and the same straight line as in Figure 4, which is just two averages connected by a line, not a fitted model. If a straight line described the trend, the gap would only wobble around zero with the seasonal swings. Instead its yearly average (thick line) forms a hump, close to zero at the two ends where the line is anchored and well above it through the 1970s-80s (roughly 90-170 megalitres). That is the same rise-plateau-fall shape as the trend itself, so a straight line doesn't describe the series. I would take the trend out first, and the seasonal pattern too (see Figure 6 and Figure 7), and look at what is left.

```
ausbeer_df$gap <- ausbeer_df$value - ausbeer_df$line
yearly_gap <- aggregate(cbind(time, gap) ~ floor(time), ausbeer_df, mean)
yearly_gap <- yearly_gap[yearly_gap$time < 2010, ] # drop the 2-quarter 2010
```

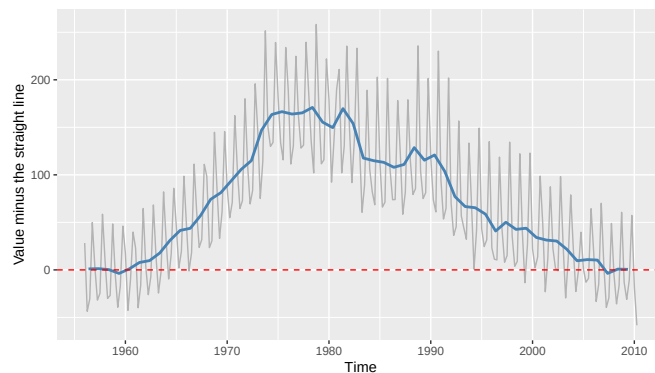


Figure 8: Gap between `ausbeer` and the straight line in Figure 4 (thin: each quarter, thick: yearly average).

A transform to stabilise the variance (log or Box-Cox) is harder to justify. The usual visual argument for a log transform is that the swings grow with the level, and neither Figure 2 nor the breakdown in section 2.4 shows that clearly: the swing grows from about 100 to 135 megalitres as the level rises to the 1980s, but shrinks to about 95 in the 2000s while the level stays around 430. So I would not transform, since the case for a log is weak and mixed.

References

- Frery, Alejandro C. 2026. *The Monte Carlo Method*. DATA301/DATA471 tutorial slides. Victoria University of Wellington.
- Hyndman, Rob J., and George Athanasopoulos. 2018. *Forecasting: Principles and Practice*. 2nd ed. OTexts. <https://OTexts.com/fpp2>.
- Hyndman, Rob J., and Yanan Fan. 1996. “Sample Quantiles in Statistical Packages.” *The American Statistician* 50 (4): 361–65. <https://doi.org/10.2307/2684934>.
- Wicklin, Rick. 2017. *Sample Quantiles: A Comparison of 9 Definitions*. SAS Blogs, The DO Loop. <https://blogs.sas.com/content/iml/2017/05/24/definitions-sample-quantiles.html>.

Appendix: Declaration of AI Use

Claude Code was used to help check and improve the R code for both exercises, to summarise and search reference material on Monte Carlo method applications, to act as a sounding board for discussing ideas, and to assist with Quarto/L^AT_EX typesetting and English language polishing. All written interpretations and analytical decisions are my own work.